WYBRANE ELEMENTY KRYPTOLOGII

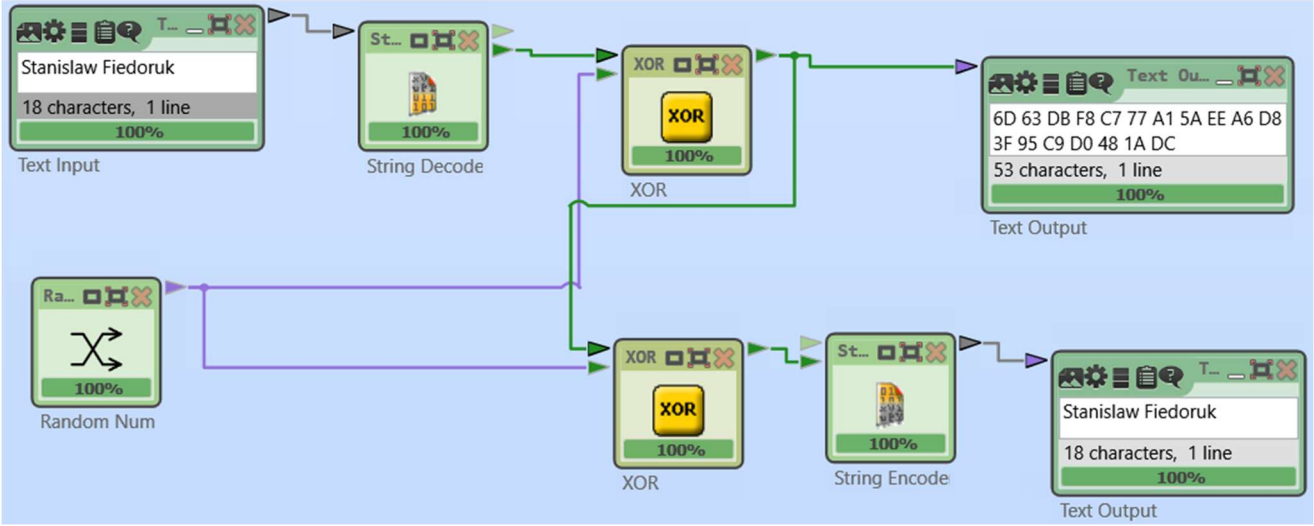
Laboratoria 3-5 – Szyfry strumieniowe

Prowadząca: mgr inż. Alicja Kida

Stanisław Fiedoruk WCY24KY1S1

1. Szyfr z kluczem jednokrotnym (OTP)

a) Szyfrowanie losowym kluczem jednorazowym



Rysunek 1: Model szyfrujący jednorazowym kluczem losowym

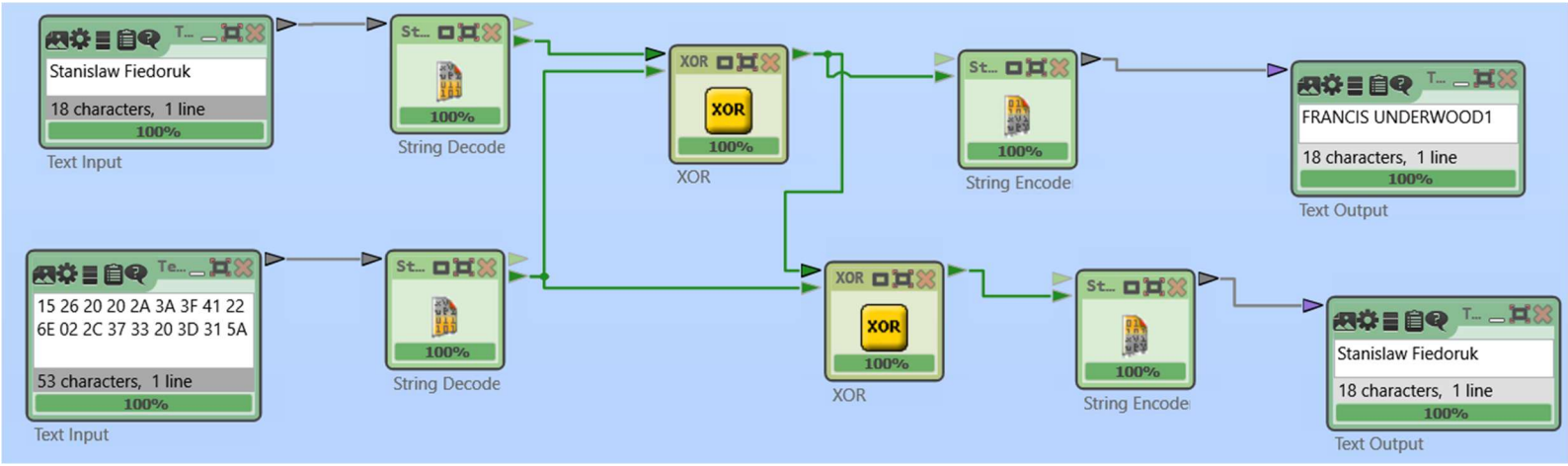
Szyfr typu OTP opiera się na losowości klucza i jest jedynym znanym szyfrem niemożliwym do złamania, ponieważ nawet znając klucz nie możemy odszyfrować wiadomości, gdyż może być ona dowolnym ciągiem znaków o tej samej długości. Proces szyfrowania to prosta operacja XOR która jest wykonywana na wygenerowanym losowo kluczu i tekście jawnym. Niezawodność polega na jednorazowości klucza, ponieważ gdyby użyć go dwukrotnie szyfrogram byłby podatny na atak. Długość klucza to 18 bajtów, taka jak długość tekstu jawnego, gdyż każdemu bitowi tekstu jawnego musi odpowiadać bit klucza, aby skutecznie wykonać XOR. [1]

Tekst jawny: Stanislaw Fiedoruk

Klucz: 3E 17 BA 96 AE 04 CD 3B 99 86 9E 56 F0 AD BF 3A 6F B7

Szyfrogram: 6D 63 DB F8 C7 77 A1 5A EE A6 D8 3F 95 C9 D0 48 1A DC

b) Model szyfrowania z kluczem dobranym tak aby otrzymać konkretny tekst



Rysunek 2: Model szyfrujący kluczem dobranym tak aby otrzymać konkretny szyfrogram

W tym przypadku klucz został dobrany tak aby szyfrogram wyglądał jak zwykły tekst. Pokazuje to, że wiadomość zaszyfrowana szyfrem OTP może stać się dowolnym innym ciągiem znaków o tej samej długości więc atakujący nie może mieć pewności, że trafił na dobrą kombinację i dobrał odpowiedni klucz.

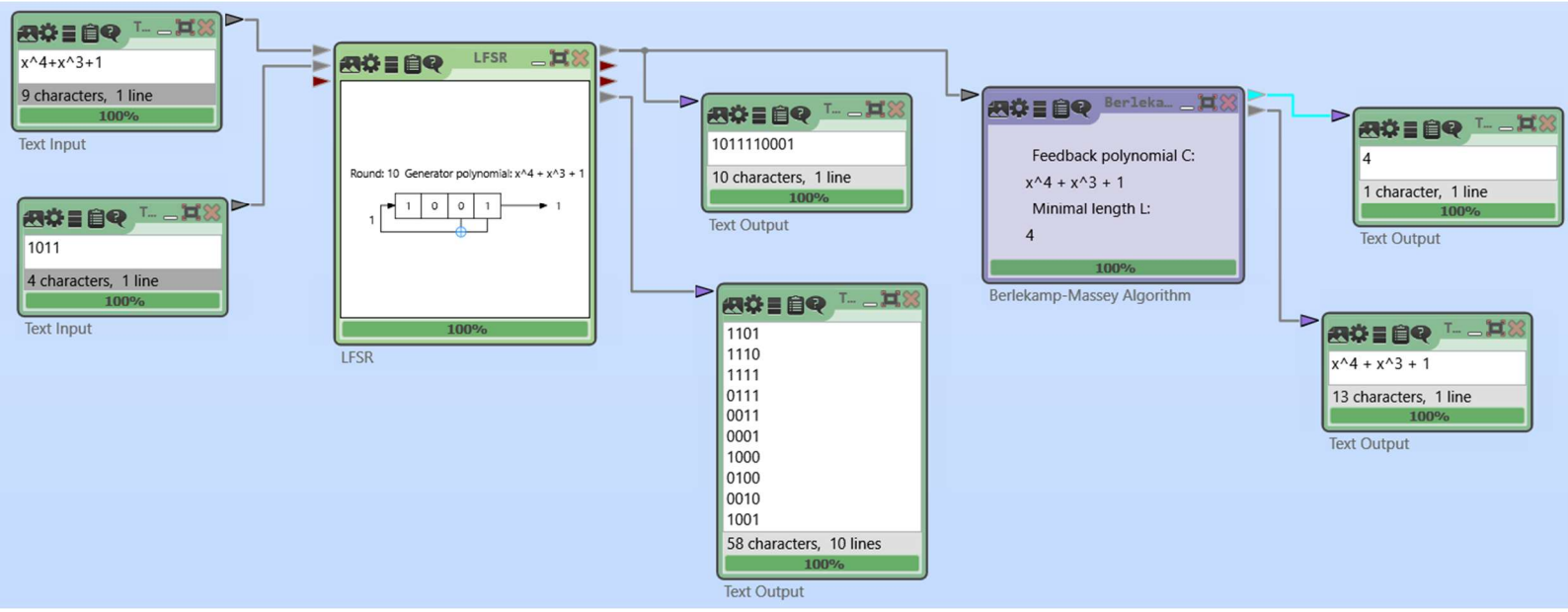
Tekst jawny: Stanislaw Fiedoruk

Klucz: 15 26 20 20 2A 3A 3F 41 22 6E 02 2C 37 33 20 3D 31 5A

Szyfrogram: FRANCIS UNDERWOOD1

2. Rejestr LFSR i atak na niego

a) Generowanie ciągu Rejestrem LFSR



Rysunek 3: Model generujący ciąg rejestrem LFSR oraz jego analiza

Rejestr LFSR (Linear Feedback Shift Register) generuje określony ciąg bitów z wykorzystaniem ziarna (wartości początkowej) i wielomianu odpowiadającego za to na których bitach nastąpi modyfikacja przy przesunięciu. Bit „wypadający" z rejestru jest modyfikowany z określonym bitem w rejestrze operacją XOR, po czym wraca na początek. [2]

Numer: 8

Wielomian: $x^4 + x^3 + 1$

Ziarno: 1011

Ciąg: 1011110001

Ziarno: 1111

Ciąg: 1111000100

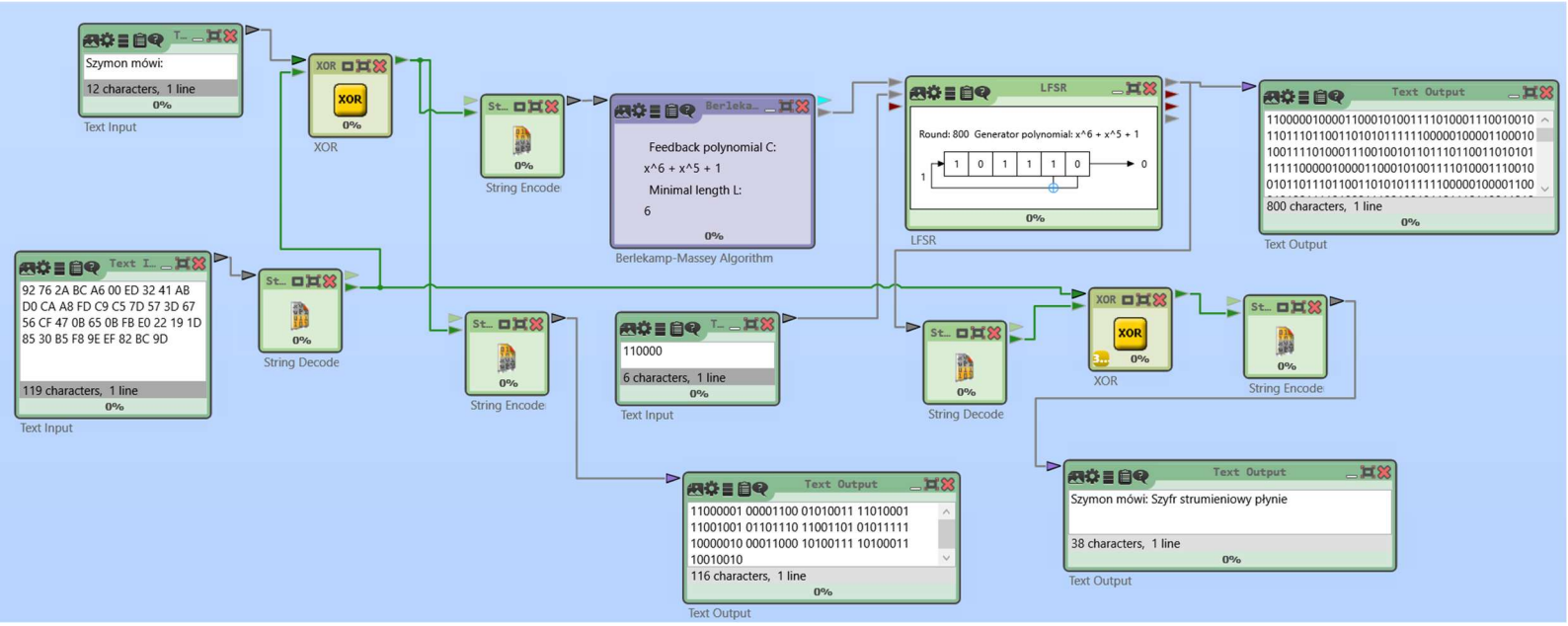
Ziarno: 0001

Ciąg: 0001001101

Okres utworzonego rejestru to $2^4 - 1$ (15). Dla każdego niezerowego ziarna okres wynosi maksimum. Gdyby ziarno było zerowe, rejestr wygenerowałby same zera

b) Algorytm Berlekamp-Massey dla danego ciągu binarnego wyznacza wielomian oraz minimalną długość ziarna użytych do jego generowania. Działa on sprawdzając od najprostszej możliwości wielomianu i ziarna i w momencie rozbieżności w ciągu są one modyfikowane aż do znalezienia jak najkrótszego dopasowania. [3]

c) Atak na szyfrogram szyfrowany z wykorzystaniem klucza generowanego przez LFSR



Rysunek 4: Model ataku na szyfrogram

Atak na szyfrogram, do którego utworzenia użyto rejestru LFSR, znając część tekstu jawnego, polega na odnalezieniu wielomianu tworzącego cały ciąg poprzez wykonanie operacji XOR na znanym fragmencie i szyfrogramie dzięki czemu otrzymujemy część ciągu jawnego który możemy przeanalizować algorytmem Berlekamp-Massey. Odnaleziony w ten sposób wielomian i ziarno (początek ciągu) używamy, aby odtworzyć resztę wiadomości.

Szyfrogram: 92 76 2A BC A6 00 ED 32 41 AB D0 CA A8 FD C9 C5 7D 57 3D 67 56 CF 47 0B 65 0B FB E0 22 19 1D 85 30 B5 F8 9E EF 82 BC 9D

Ziarno: 110000

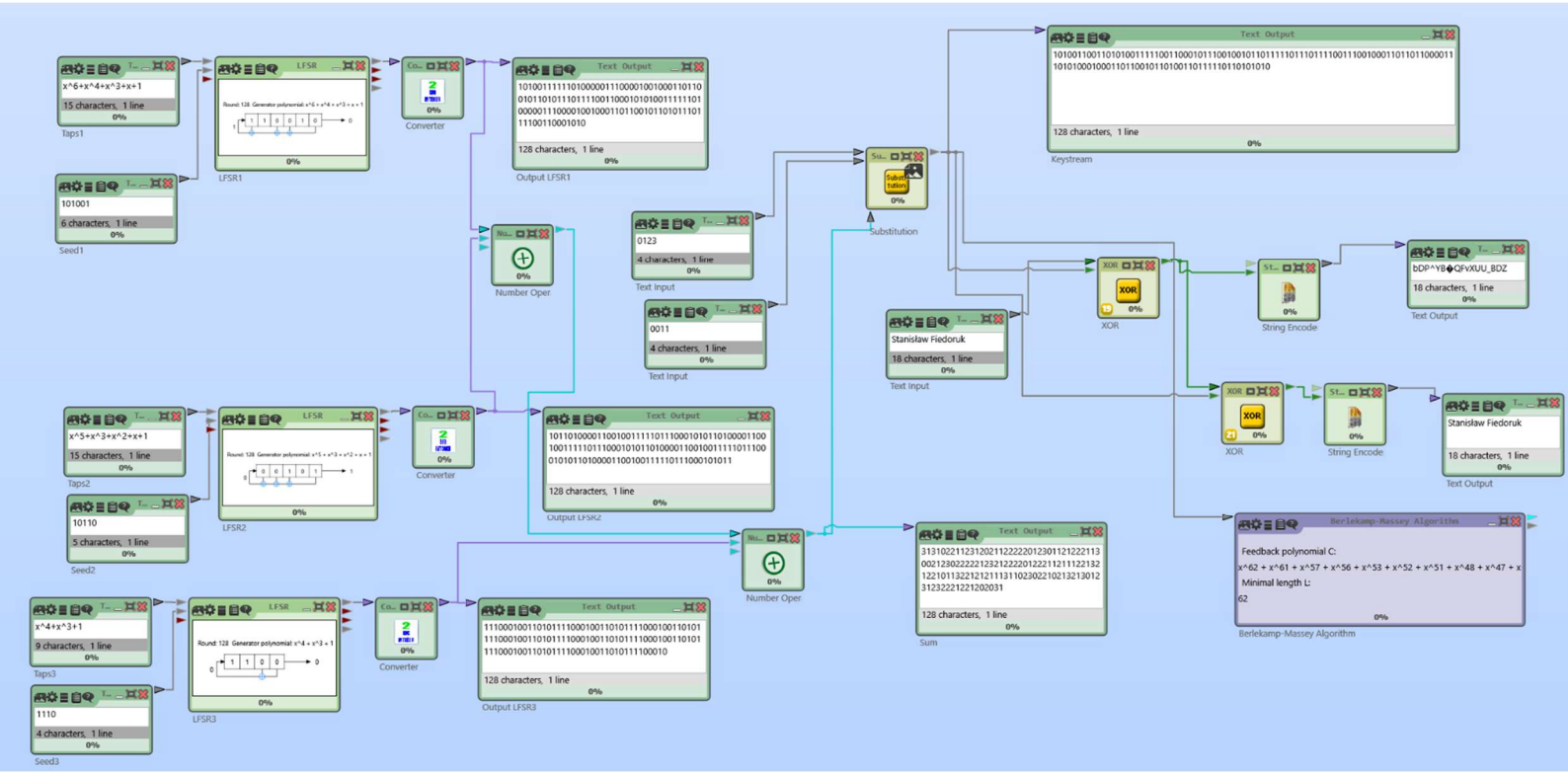
Tekst jawny: Szymon mówi: Szyfr strumieniowy płynie

Wielomian: $x^6 + x^5 + 1$

Początkowe bajty strumienia klucza: C1 0C 53 D1 C9 6E CD 5F 82 18 A7 A3 92

3. Generator progowy na bazie rejestrów LFSR

a) Generator progowy na bazie trzech rejestrów LFSR



Rysunek 5: Model szyfrowania z generatorem progowym z wykorzystaniem 3 rejestrów LFSR

Generator progowy działa na zasadzie połączenia nieparzystej liczby rejestrów liniowych (tutaj 3 rejestry LFSR) które równolegle generują ciągi binarne. Bity ciągu całego generatora są wybierane demokratycznie, czyli o tym czy wystąpi 0 czy 1 decyduje to którego znaku jest najwięcej. Za pomocą ciągu z generatora zaszyfrowano tekst jawny szyfrowaniem XOR.

Tekst jawny: Stanisław Fiedoruk

Klucz: 101001101101110

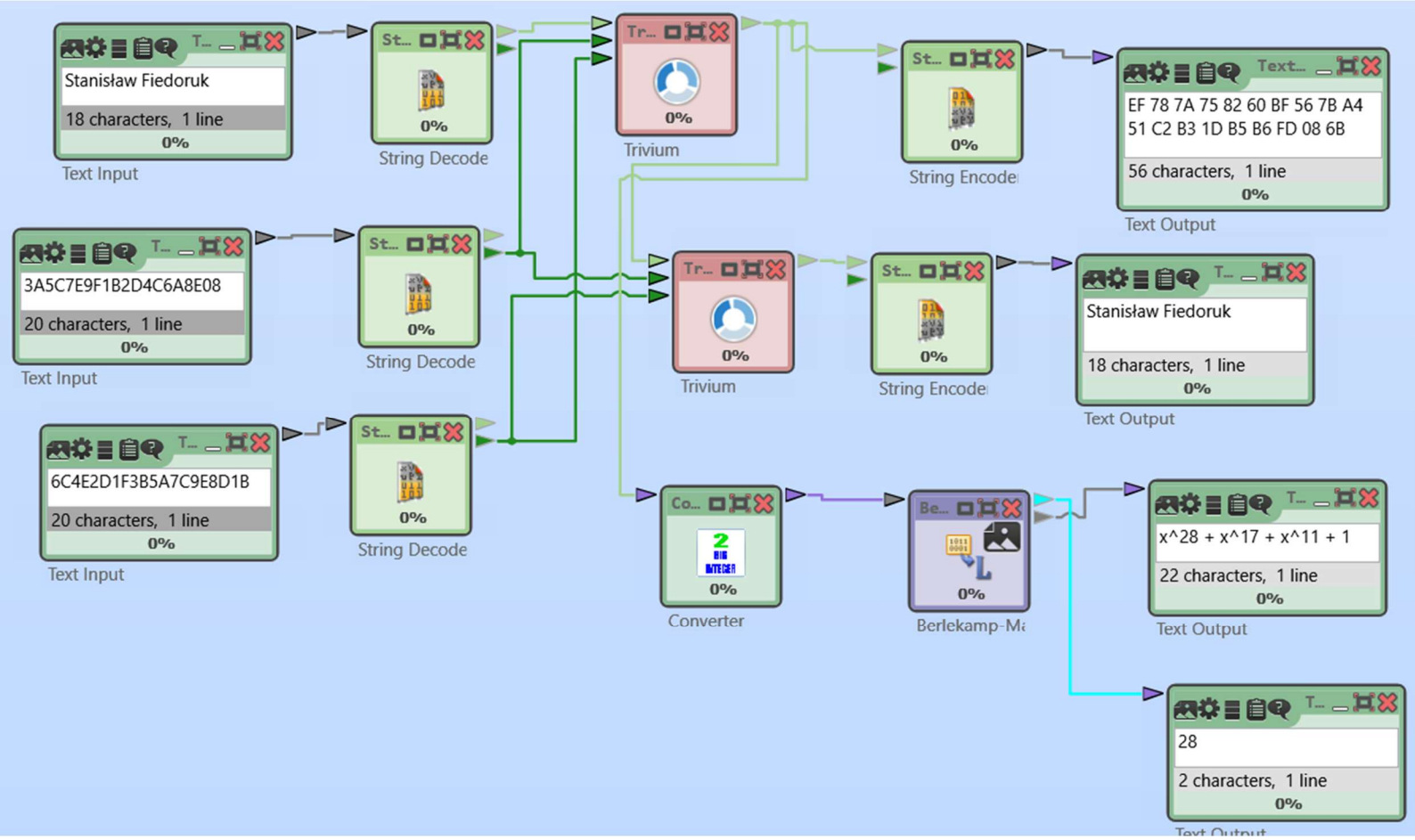
Wielomiany: $x^4 + x^3 + 1,\ x^5 + x^3 + x^2 + x + 1,\ x^6 + x^4 + x^3 + x + 1$

Szyfrogram: 62 44 50 5E 59 42 F4 B2 51 46 11 76 58 55 55 5F 42 44 5A

b) Analiza algorytmem Berlekamp-Massey z powodu nieliniowości generatora nie przynosi żadnych użytecznych rezultatów, podała ona wielomian 62 stopnia, gdzie tak naprawdę użyto wielomianu o maksymalnie 6 stopniu do wygenerowania analizowanego ciągu.

4. Szyfr Trivium

a) Szyfrowanie i deszyfrowanie z wykorzystaniem Trivium



Rysunek 6: Model szyfrujący tekst poprzez Trivium oraz analizujący powstały szyfrogram

Szyfrowanie szyfrem Trivium polega na kombinacji trzech rejestrów przesuwających o łącznej długości 288 bitów. Na wejściu podane są tekst jawny i wektor inicjalizujący. Samo generowanie podzielone jest na dwie fazy. Inicjalizacji, w której bity krążą po rejestrach 1152 razy wykonując operację matematyczne i nie emitując żadnego wyjścia. W kolejnej fazie bity są pobierane z konkretnych pozycji i przechodzą przez układ złożony z bramek logicznych po czym trafiają na wyjście. Główną siłą tego szyfrowania jest jego nieliniowość, przez którą jest niemożliwy do analizy algorytmem Berlekamp-Massey. [4]

Tekst Jawny: Stanisław Fiedoruk

Klucz: 3A5C7E9F1B2D4C6A8E08

Wektor Początkowy: 6C4E2D1F3B5A7C9E8D1B

Szyfrogram: EF 78 7A 75 82 60 BF 56 7B A4 51 C2 B3 1D B5 B6 FD 08 6B

b) Wielomian będący wynikiem analizy jest o wiele wyższego stopnia od poprzednich co wynika z nieliniowości, jego długość będzie zmieniać się ze zmianą długości ciągu wynikowego z Trivium, ponieważ algorytm będzie sztucznie zmieniać jego długość przez niegodność ze źródłem.

   Wielomian: $x^{28} + x^{17} + x^{11} + 1$

c) Inne szyfry strumieniowe dostępne w programie CrypTool2: RC4, Salsa 20, Rabbit, Mickey 2

Bibliografia:

[1] https://en.wikipedia.org/wiki/One-time_pad

[2] https://en.wikipedia.org/wiki/Linear-feedback_shift_register

[3] https://en.wikipedia.org/wiki/Berlekamp%E2%80%93Massey_algorithm

[4] https://cr.yp.to/streamciphers/trivium/desc.pdf